

实验一 GNU Radio 流程图编程

实验目的：

学习 GNU Radio 流程图编程。了解 GRC，找到自己需要的模块，修改模块的参数，设计第一个信号流程图，进行 Generate Options 设置，查看输出。

实验原理：

GRC 是基于模块的仿真实验工具。类似于 MATLAB 中的 SIMULINK 工具。基本使用方法就是先将需要用的模块放到窗口。再设置各个模块的参数，然后连接各个模块，绘制信号流程图，之后生成流程图，仿真运行。

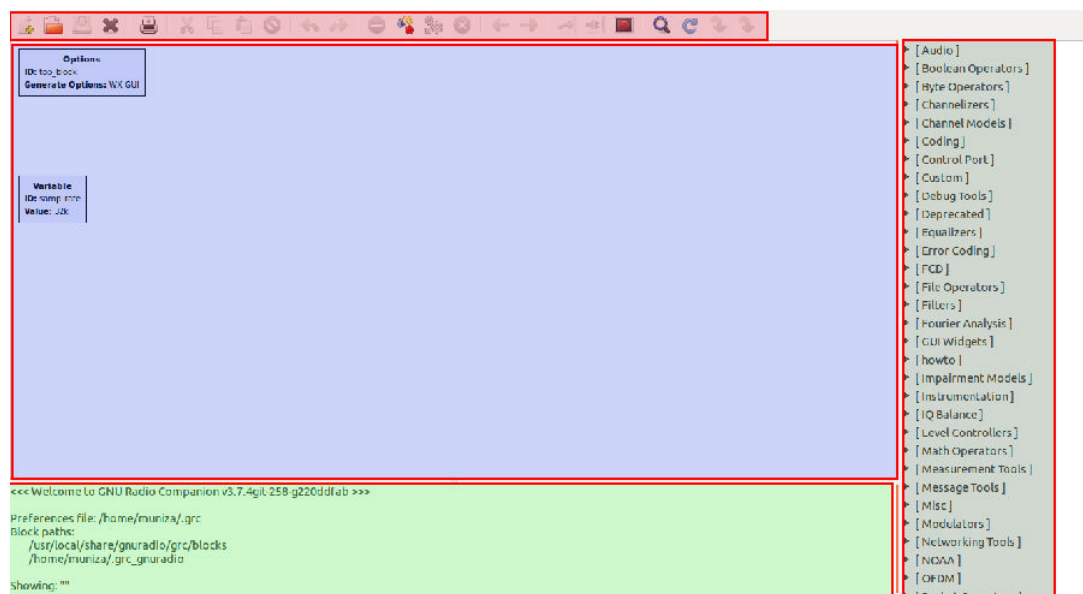
实验步骤：

登陆：<http://ilab.xidian.edu.cn/>

1. 开始了解 GRC

GNU Radio 是一个各种信号处理模块的“容器”。在这里，我们将从一个简单的例子讲起，展示如何使用 GNU Radio Companion(GRC)。我们需要清楚的是，GRC 是允许我们创建 Python 文件图形的，不过流程图的方式更加简化了 GNU Radio 的使用。

首先我们来看一下 GNU Radio Companion 的界面。它包括 4 个区域：模块库、工具栏、函数输出窗口和工作区域。



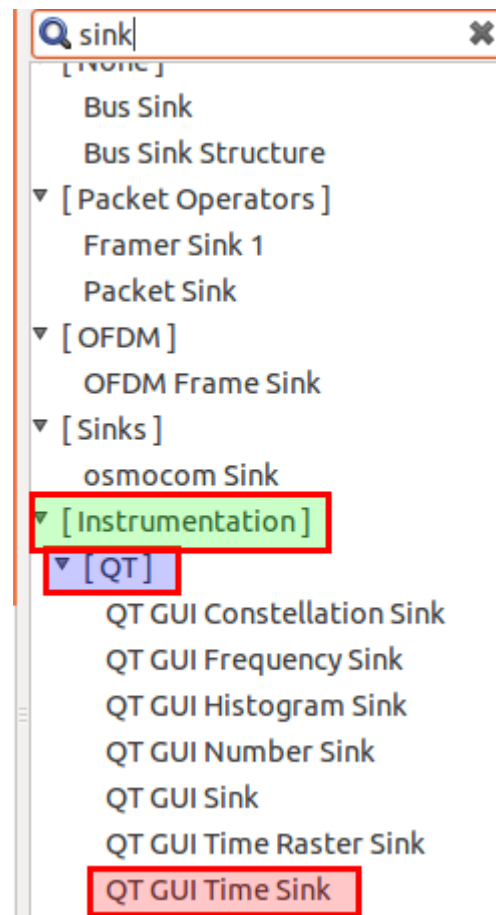
打开 GRC 的方式，

1>在 VNC 终端输入命令：`gnuradio-companion`

2>VNC 桌面下：Applications->开发->GUN Radio Companion

2. 找到自己需要的模块

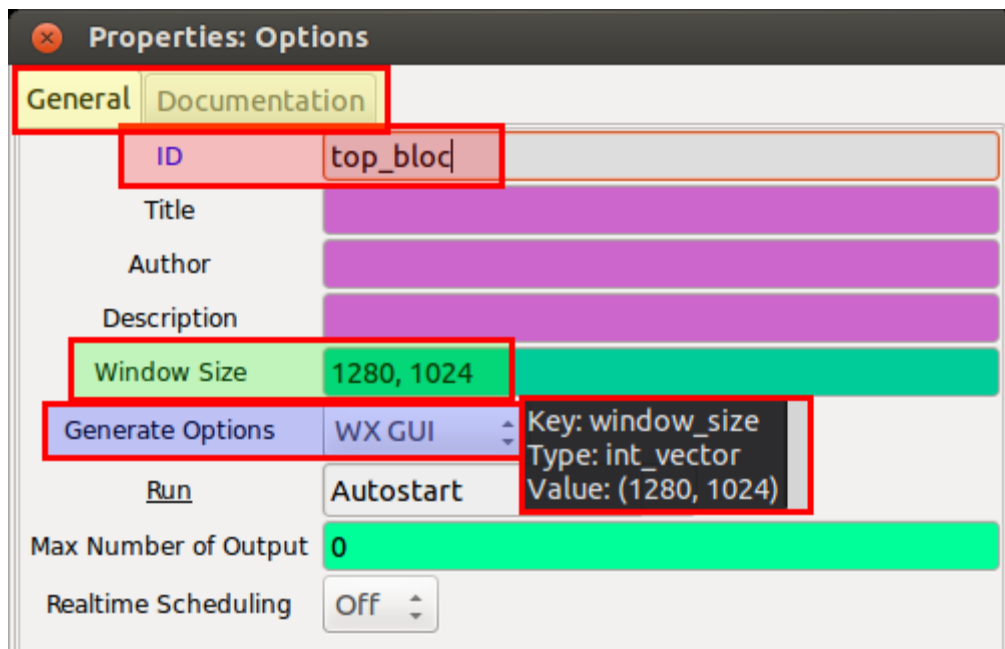
在右边的模块库里面有各种不同类型的信号处理模块，包括了 GNU Radio 的标准模块和我们初始化的模块。假设需要一个产生信号的模块，可以看到有一个叫做 **Waveform Generator** 的类型，打开就可以看到各种产生信号的模块了。假设需要一个展示波形图，但是又不确定哪种模块是效果最好的，需要使用搜索的功能了，按 **Ctrl+F**，或者点击搜索按钮，然后输入模块的关键字 **sink**，可以看到很多关于 **sink** 的模块：



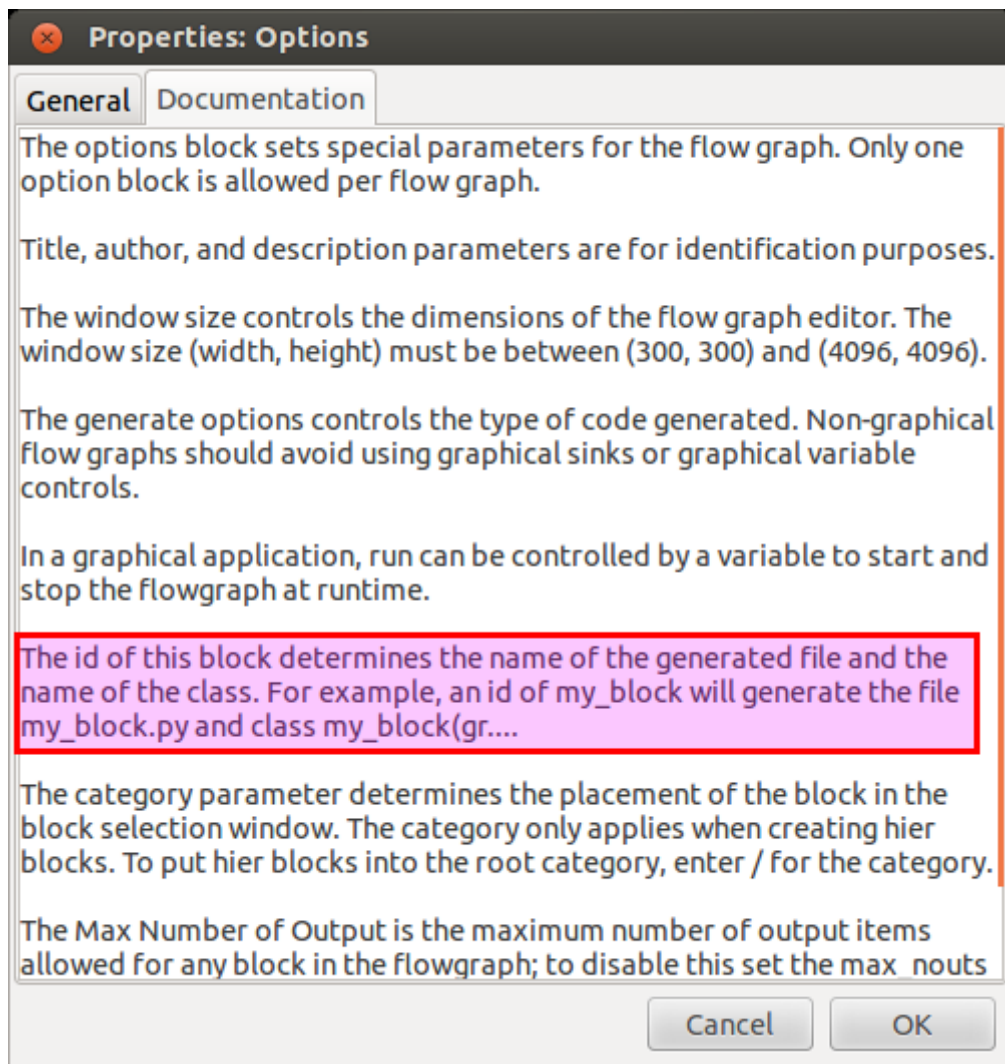
现在添加一个叫 **QT GUI Time Sink** 的模块。可以通过双击或拖动的方式。

3. 修改模块的参数

工作区域里面包含各种信号处理模块和变量。双击打开模块，设置它的参数属性。如下图：

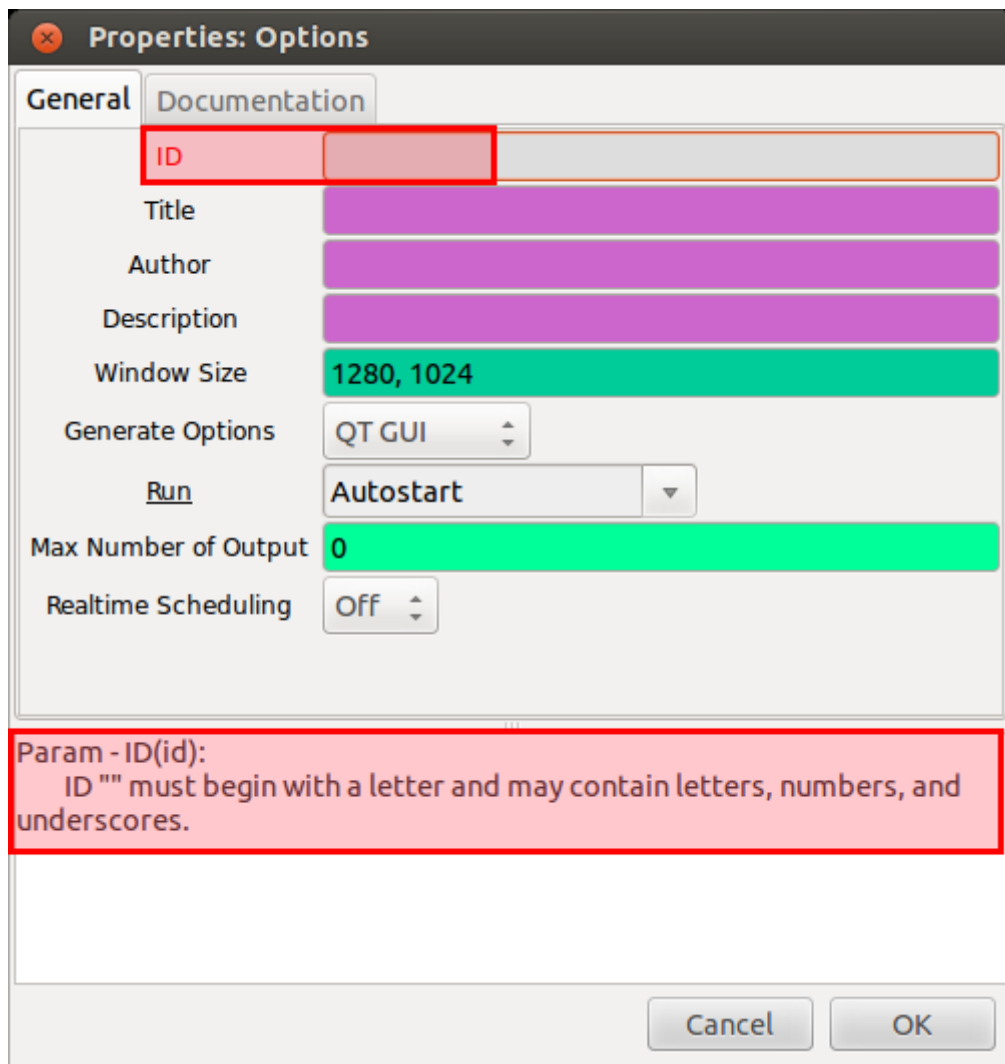


在不同的应用系统中，可能需要修改这些默认的参数。把原来的名字去掉，可以看到 ID 字样变成蓝色。这种颜色表明信息已经被修改，但是还没有保存。再尝试一下修改 windows size 为 300,300，单击 OK。然后可以看到尺寸的改变。然后，看到 documentation:



看用颜色标记的字段，知道 ID 用来确定 python 文件名字和相关类的名字。

然后，删除掉 ID 里面的字符串，看到最底下出现了红色的报错信息，同时，ID 也变成了红色，这样我们能够很方便的确定错误的所在。

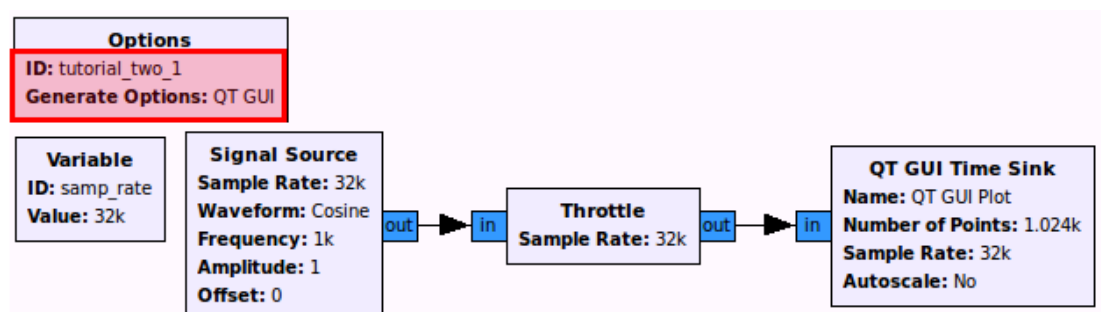


为了方便，例如取 ID 的名字为“tutorial_two_1”，同时将 Generate Options 改成“QT GUI”，因为采用的是 QT GUI sink，而不是 WX GUI sink。在最新版的 GNU Radio3.8 版本，默认的是 QT GUI。GRC 是一个图形化的界面，它是基于 python 环境的，所以当我们执行一个流程图时，实际上我们真的运行的是一个 python 程序。ID 是用来命名 python 文件名，与 GRC 文件保存在同样的路径下。默认情况下，ID 是 top_block，所以它创建一个为 top_block.py 的文件。更改 ID 使我们可以更改保存的 python 文件名，以更好进行文件管理。

注意的另一个关键是接口不同的颜色。这些实际上对应于不同的数据类型，我们将在本讲义后面稍后解释。

4. 第一个信号流程图

现在我们了解如何找到块，如何将它们添加到工作区，以及如何编辑块属性，我们尝试建立一个将信号输出到示波器的流程图，注意模块之间的数据类型的匹配：



注意到有一个叫 `throttle block` 的模块：在本教程后面的更详细的解释是什么。现在，可以了解的是这个模块确保流程图不消耗 100% 的 CPU 周期，不然你的电脑可能反应不过来。

在此之前，我们先来看一下工具栏。



这里主要是一些流程图软件执行的一些命令，如新建，打开，保存等。让我们开始我们的流程图，我们给它取名叫做 `tutorial_two`。这里有几个重要的工具，`Generate flowgraph`, `excute flowgraph`，分别是产生 GRC，和运行 GRC。在菜单 `help` 里面的 `type`，我们可以看到每种数据类型对应的颜色。

5. Generate Options 设置

`Generate Options` 有两种常用的设置 `QT GUI` 和 `WX GUI`。最常见的错误就是设置的 `Generate Options` 与实际我们用的不匹配。具体来讲，就是我们 `Generate Options` 设置的是 `QT GUI`，但是我们构造的流程图却是 `WX GUI` 我们会的到如下的报错信息：

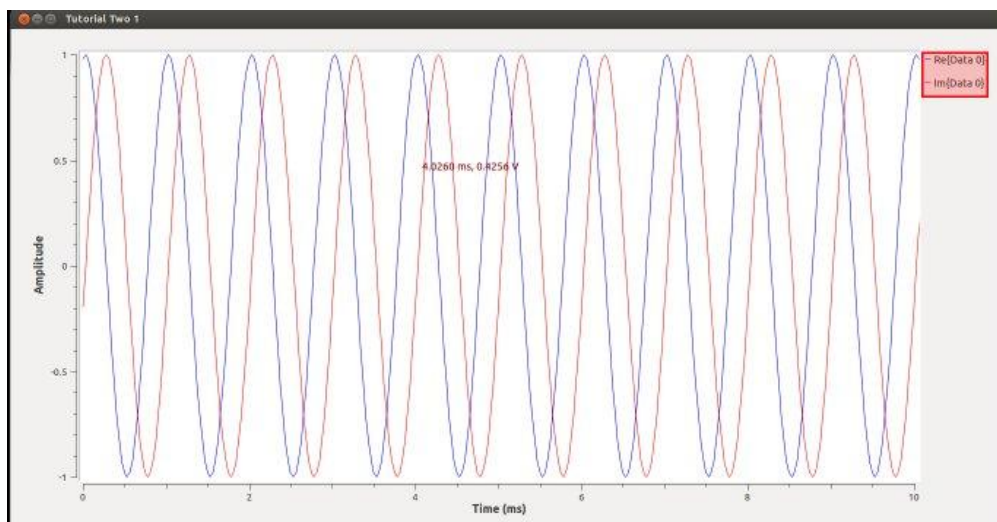
```
Executing: "/home/muniza/Documents/grc_files/gnuradio_tutorials/tutorial2/grc_files/tutorial_two_1.py"
Traceback (most recent call last):
  File "/home/muniza/Documents/grc_files/gnuradio_tutorials/tutorial2/grc_files/tutorial_two_1.py", line 106, in <module>
    tb = tutorial_two_1()
  File "/home/muniza/Documents/grc_files/gnuradio_tutorials/tutorial2/grc_files/tutorial_two_1.py", line 55, in __init__
    self.GetWin(),
  File "/usr/local/lib/python2.7/dist-packages/gnuradio/gr/top_block.py", line 101, in __getattr__
    return getattr(self, tb.name)
AttributeError: 'top_block_sptr' object has no attribute 'GetWin'
```

>>> Done

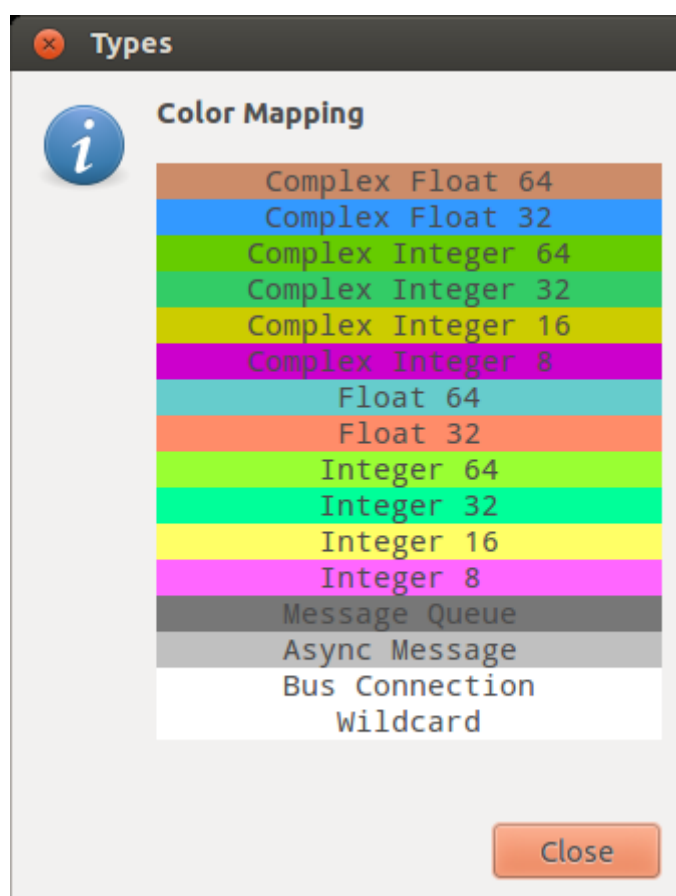
反过来的话，也会得到类似的报错信息。所以，我们必须首先确定自己选用的是 `QT GUI` 还是 `WX GUI`。

6. 查看输出

让我们点击 Execute 按钮启动程序，我们可以看到如下的波形：

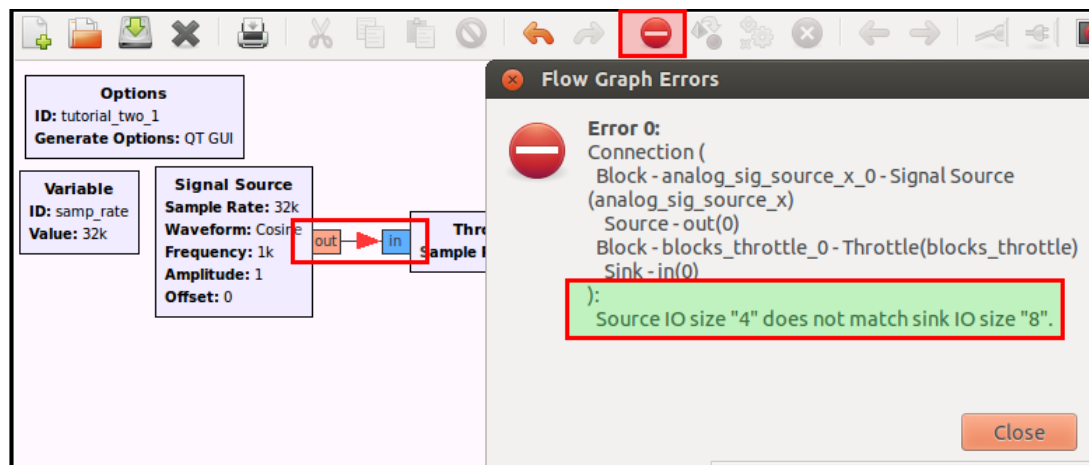


这是一个复数类型的波形。我们简单化一些，选用别的数据类型。关掉程序，我们打开菜单 Help->Type，可以看到如下的数据类型的代表颜色：

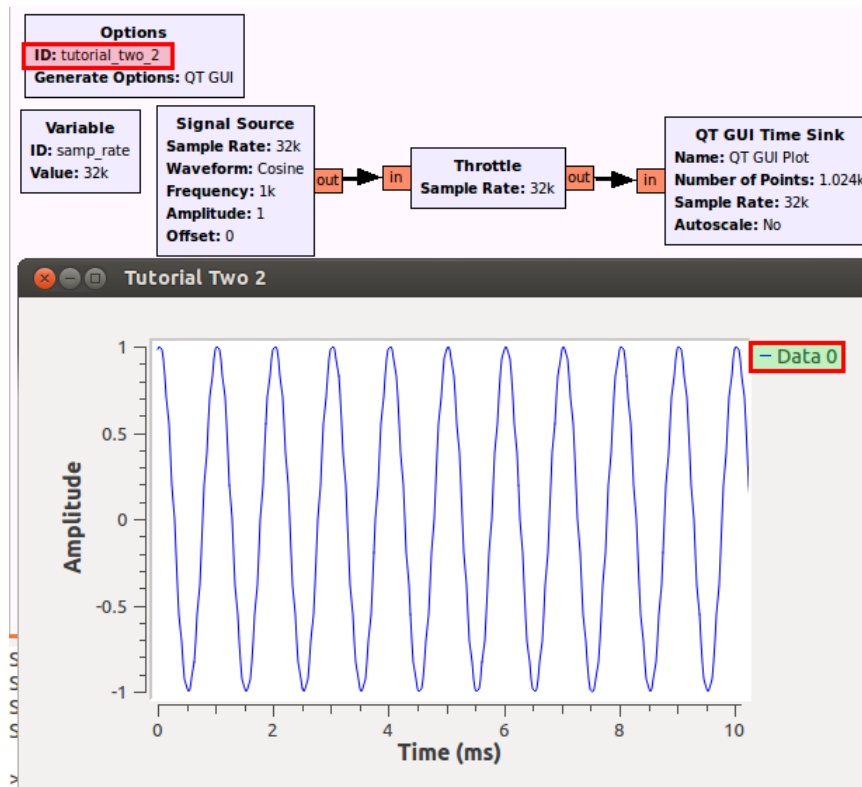


在这里，我们基本上可以找到所有的数据类型。我们看到我们的流程图的接口是蓝色的，很明显这是 32 位浮点型组成复数形式，现在我们可以解释刚才

输出有两个波形的情况了，Time Sink 输入一个复数，然后输出它的实部和虚部。现在，我们尝试修改以下信号源的数据类型，将其改为浮点型，然后我们看到它的接口编程橘黄色的，当我们把它和 Throttle Block 连接时，看到有红色的报错，我们点击工具栏的红色的横杆，可以看到具体的报错信息：



报错信息显示，数据长度不匹配。这是由于我们的数据类型不匹配的原因。GNU Radio 不允许不同的数据类型之间直接连接。我们把所有的模块都改成浮点型的，然后再次运行，可以看到如下的波形：



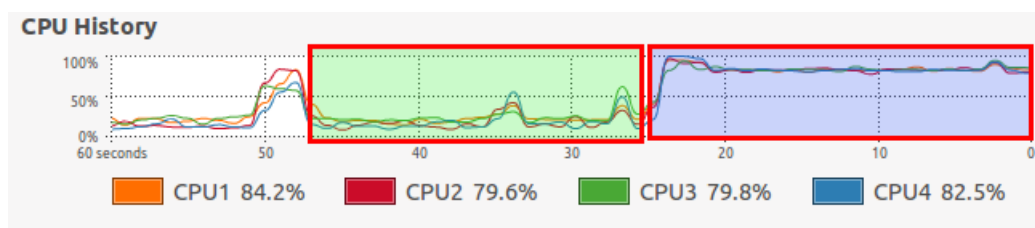
现在我们可以看到，只有一个波形输出了，我们可以尝试用鼠标放大或者缩小。

7. 使用 GNU Radio Companion

现在我们已经能够创建自己的流图了，我们进一步的来学习一些 GNU Radio Companion 有关知识。

7.1 Throttle Block

首先，我们来讨论下 Throttle Block，之前在我们的流图有用到过。下图是添加 Throttle 模块和不添加 Throttle 模块的 CPU 的使用率对比：

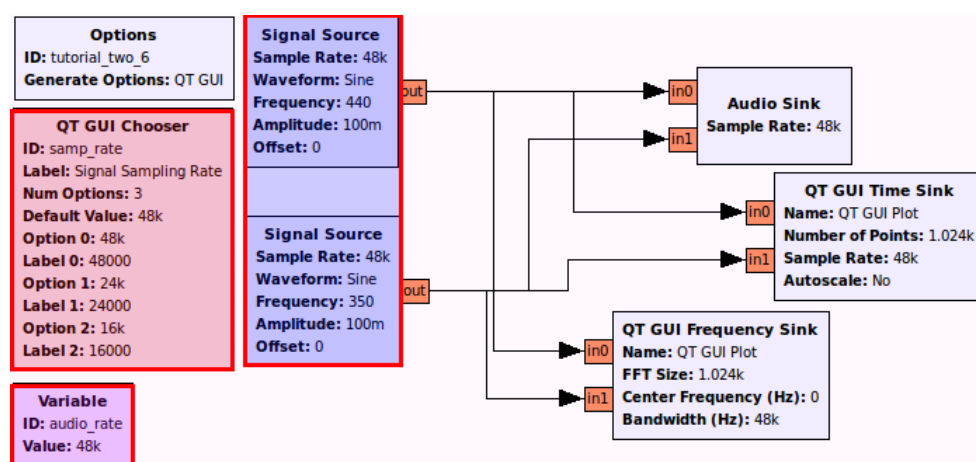


我们可以看到，当流图没有连接硬件也没有连接 Throttle 的时候，CPU 基本是满负载运行的。在一个流图中我们只需要一个 Throttle 就可以了，不管它是有几个输入输出。我们可以认为 Throttle 起到限速的作用，设置高的速率，流图执行快，设置低的速率流图执行慢。

当有硬件连接的时候，则不需要 Throttle 模块，因为硬件本身已经对速率有了限制。

7.2 关于采样率

我们已经知道设备需要设定特定的采样率来实现某些特定的功能，现在我们来自己创建一个 GRC，来看看不同的采样率之间有什么区别：

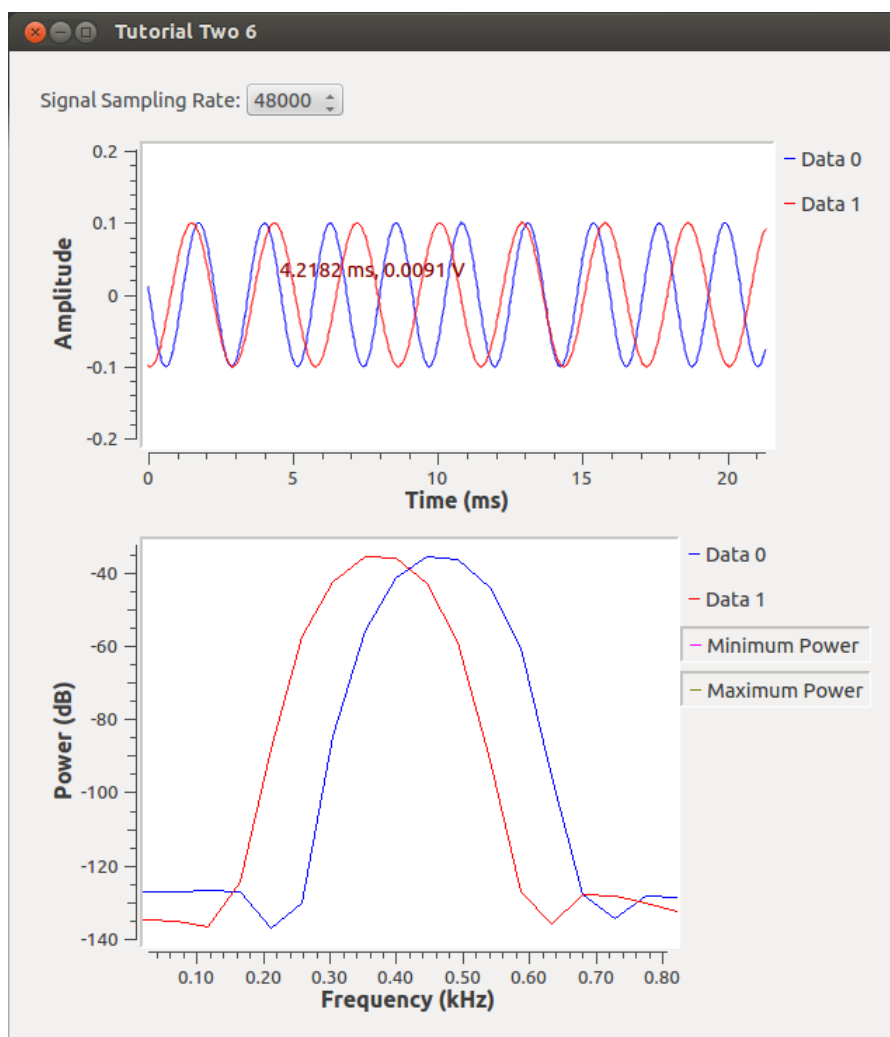


根据上图配置好参数：

- QT GUI Chooser 模块设置 3 个采样速率 48000/24000/72000，ID 改为 samp_rate

- 所有的 source 模块的 Sampling Rate 改为 samp_rate
- Variable 模块设置音频模块采样率为 48000，ID 改为 audio_rate
- 所有的 sink 模块的 Sampling Rate 改为 audio_rate
- 信号源是两个频率分别为 440Hz 和 350Hz 的正弦信号，幅度为 0.1
- 所有 sink 模块的 Number of inputs 设置为 2

在这里，我们可以看到没有 Throttle 模块，这是因为这里有音频硬件 Audio sink。点击运行，设置 sample rate 为 48e3，就会听到熟悉的电话的拨号音（在线实验平台不支持声卡），再看一下 fft，确实有两个分别为 440Hz 和 350Hz 的频段。

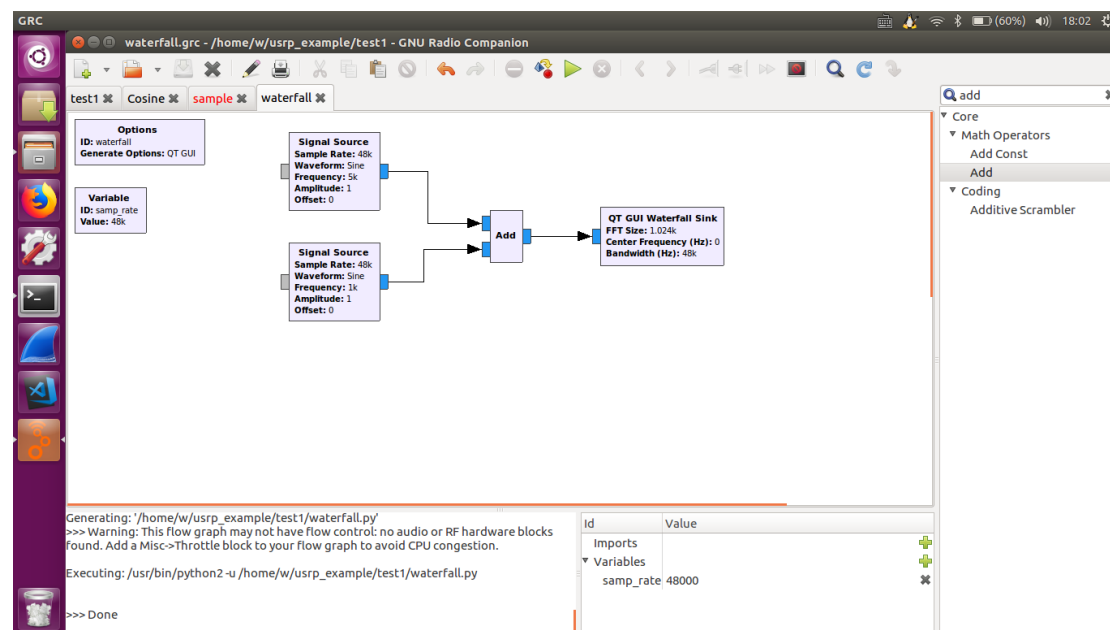


尝试下，当改变 sampling rate 比 audio sampling 低的时候，看能否还能听到原拨号音。再尝试下，当改变 sampling rate 比 audio sampling 高的时候，看能否还能听到原拨号音。一般情况下，应该听到变调的拨号音，这是因为声卡的速率和我们采样的速率不匹配。所以，任何时候，都应该设置正确的采样率。因

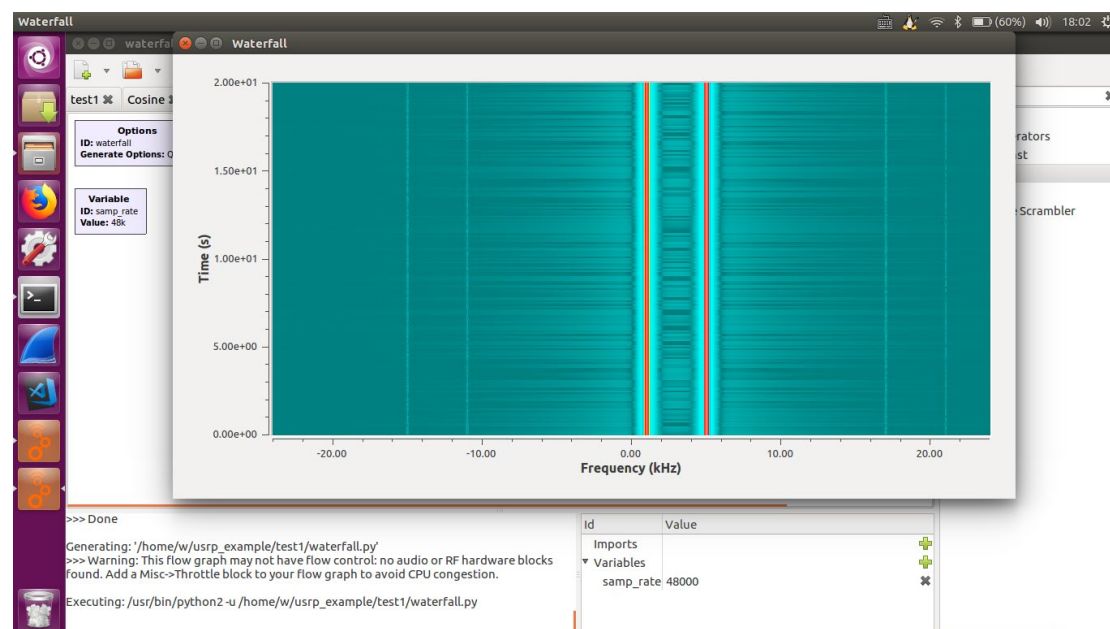
为我们的在线实验平台采用远程桌面，没有声卡支持，感兴趣的同学可以在本机安装 Gnuradio 软件进行验证。

8. 瀑布图

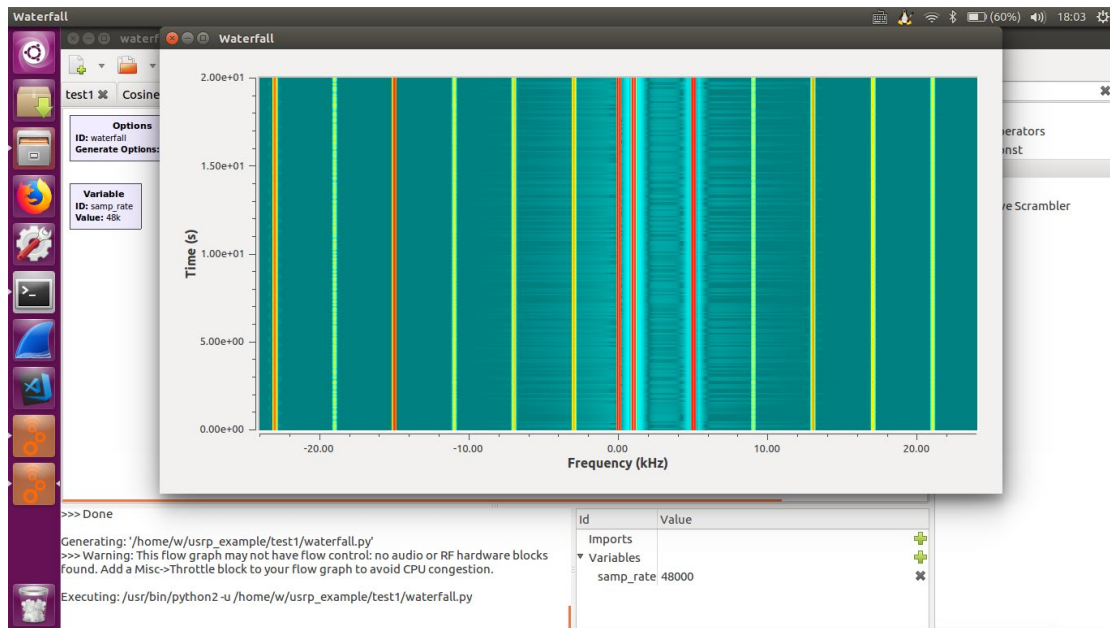
现在来看由 QT GUI Waterfall Sink 模块组成的正弦信号瀑布图，并了解瀑布图的含义。



请参考上图设置好参数，点击运行，可以看到：



将任一信源的波形改成三角波后，可以看到：



自行了解瀑布图并解释上述变化的原因。